

Technical Companion & Decision Log

A running record of the specific choices in this pipeline and the reasoning behind them. Not a primer on what a PLM is — this documents what each model does differently, what each feature representation concretely computes, why the splits are built the way they are, and how to read every evaluation figure in terms of the TEM-1 functionality problem. Read it alongside the literature review and the feature-representation catalog.

1. The specific models, and why they differ

All the models below read a sequence and nothing else. What separates them is training objective, training data, size, and what you can pull out of them. This section is about the practical differences that change what a feature *means*, not the marketing.

ESM-2 vs ESM-1v — same idea, different priorities

Both are masked language models from the ESM line, so both give you the same three readouts (a masked-marginal score, a 20-way per-position distribution, and embeddings). The difference is what they were tuned for.

- **ESM-1v** was built and validated *specifically for zero-shot variant effect prediction*. It is an ensemble of five models, and its whole reason to exist is that $\log P(\text{mutant}) - \log P(\text{wildtype})$ correlates with mutational effect. If you want the cleanest single zero-shot score, this is the reference.
- **ESM-2** is a general-purpose model trained later, larger, and better. It was not variant-effect-specialized, but its representations are stronger across the board, which is why a folding head (ESMFold) could be bolted onto it. For *embeddings* (Representation 2) ESM-2 is usually the better feature source; for a *pure zero-shot scalar* ESM-1v was purpose-built.

Practical consequence for us: we will likely use **ESM-2 650M** as the embedding/representation workhorse and keep **ESM-1v** as the specialized scalar scorer, then let the sub-study tell us which actually wins on TEM-1.

Why ESM-2 stays even though ESMFold is a "folder"

This trips people up. ESMFold is a structure predictor — it outputs 3D coordinates. But ESMFold is *ESM-2 with a folding head on top*. The ESM-2 trunk reads the sequence and produces internal representations; the folding head turns those into a structure. We never use the folded structure as a feature (that would be a structure-derived pipeline). We use the ESM-2 trunk's own outputs — the logits and embeddings that come straight off the sequence, before any folding. So "ESMFold is in the family" and "we only feed sequence-derived features" are both true at once: the trunk is sequence-in, the

coordinates are what we ignore.

ESM C (ESM Cambrian)

Newer EvolutionaryScale family, optimized to give strong representations at lower compute. Attractive because we run on CPU and cost matters. Caveat: it shipped as a versioned model release, not a peer-reviewed paper, so treat its reported numbers as vendor-stated until our own benchmark confirms them. It is a candidate, not a default.

ProtT5 and VESPA — the non-ESM comparators

- **ProtT5** is an encoder-decoder (T5) trained on protein sequences. Different architecture, different training data than ESM, so its embeddings are *partially independent* evidence — which is exactly why concatenating it with ESM (Representation 6) can help.
- **VESPA** is not a general LM; it is a variant-effect predictor *built on top of* ProtT5 embeddings plus a conservation head. It is the closest thing to a purpose-built competitor for our exact task, so it is the comparator to beat with a custom classifier.

CARP, Ankh, ProteinBERT, UniRep — the cheap seats

These matter because of the CPU constraint. **CARP** is convolutional (no attention), so it is cheaper to run and its per-position features rival transformers on several tasks. **Ankh** is a compute-optimized transformer. **ProteinBERT** and **UniRep** are older and smaller. We keep them as fallbacks if the big ESM checkpoints are too slow to score the full dataset locally.

The structure predictors (AlphaFold2/3, OmegaFold) — reviewed, not fed

They take sequence in, so the literature review analyzes them. But they output coordinates, and turning coordinates into a classifier feature requires a structural-feature step — which leaves the "direct-from-sequence feature" regime this benchmark is built on. So they appear in the review and are *not* feature sources in the grid. The one seat that stays is ESMFold's trunk, for the reason above.

2. What each feature representation actually computes

The exact definitions live in `02_feature_representations.md`; this is the intuition for the ones that get confused with each other.

Masked-marginal score (Rep 1) — one number

Mask the position, ask the model "what belongs here?", compare its answer for the wild-type residue vs. the mutant. If the model strongly prefers wild-type, the mutation gets a big negative score (predicted

damaging). It is one number per variant. It throws away everything except that one comparison — which is its weakness.

Per-position surprisal vector (Rep 3) — twenty numbers

Same forward pass as Rep 1, but instead of collapsing to one number you keep the model's *entire* opinion about the position: its predicted probability for all 20 amino acids. This tells you the *character* of the position — "only hydrophobics allowed here", "anything goes here" — not just the score of the one mutation. Almost free once you have Rep 1, and much richer.

Site surprisal + one-hot (Rep 4) — forty numbers

Rep 3 (the position's 20-way tolerance profile) plus a 20-way one-hot of *which* residue the mutation introduced. Now the model input encodes both *where* (the profile) and *what* (the substitution), so a decision tree can learn rules like "this position only tolerates small residues, and this mutant is bulky → damaging." This is the representation best matched to tree learners, which is why the project calls it out by name.

Mean-pooled embedding (Rep 2) — ~1280 numbers

Run the whole mutant sequence through the model, average the per-residue hidden states into one long vector. This is the model's holistic take on the entire variant sequence, catching distributed effects a single-position score cannot. The cost: one forward pass *per variant* (4,783 of them), versus one pass *per position* (263) for Reps 1/3/4. It is the expensive option and the highest-dimensional one.

Why masked-marginal is cheap and embeddings are expensive

TEM-1 has 263 positions. Masked-marginal only needs one forward pass *per position* (mask it, read the distribution), and all 19 possible mutations at that position reuse the same pass — 263 passes total. Embeddings need the *mutated full sequence* each time, so every one of the 4,783 variants is its own forward pass. That ~18× difference is the whole reason the sub-study checks whether the cheap representations are good enough before committing to embeddings.

Physicochemical (Rep 7) and why we add charge

PLM surprisal is good at conservation/stability and weaker at enzyme-specific catalysis. Amino-acid charge, hydrophobicity, and volume are free to compute from the sequence and carry a *different* kind of signal — the direct biophysical consequence of the swap (a charge reversal in a salt bridge, a bulky residue jammed into the core). Adding them gives the classifier an axis the PLM under-weights. This is why you specifically asked to include charge: it is readily available and orthogonal.

Identity one-hot (Rep 8) — the control that proves the point

Encodes only "residue X became residue Y (at position P)". No biology. It is not meant to win. Its job is to be the floor: if it scores well on the random split and *collapses* on the contiguous split, that gap is the evidence that it was memorizing positions, not learning biology — and that the PLM features, which hold up on the contiguous split, carry something real.

3. The three splits, and why contiguous is the honest one

Every model is evaluated three ways. The splits are not interchangeable, and quoting the wrong one is how variant-effect models get oversold.

Random split. Variants are shuffled and split without regard to position. The problem: the same *position* appears in both train and test (e.g. position 104 with mutation A in train, position 104 with mutation B in test). A classifier can memorize "position 104 is intolerant" and score any mutation there correctly without understanding the mutation at all. This is **position leakage**, and it inflates accuracy. Random is the optimistic number.

Modulo split. Positions are assigned to folds by index modulo k (position $\% k$). This spreads positions across folds systematically and breaks up the exact-position overlap somewhat, but adjacent positions (which are structurally correlated) can still land in different folds, so some leakage remains. It sits between random and contiguous.

Contiguous split. Whole *stretches* of the protein are held out — e.g. train on residues 24–200, test on 201–286. Now the model must score mutations in a region whose positions it has *never seen*. There is no position to memorize. This is the honest analog of the clinical situation: a novel resistance gene is an unseen region. **Contiguous is the headline number**, and we always report all three side by side so the "leakage tax" (how much accuracy evaporates from random $\rightarrow$ contiguous) is visible.

The project's central claim lives in this comparison. If PLM features stay strong on contiguous while identity one-hot falls apart, that is the proof that PLMs learned transferable biology and one-hot learned patterns. If a model only looks good on random, it has learned nothing deployable.

One hard rule that sits on top of all splits: the 13 wet-lab validation variants are sealed out of *everything* — training, tuning, split construction, model selection. They are scored exactly once, at the very end, after the model is locked. Any variant that leaks into training would make its later "prediction" meaningless. Every processed dataset carries an assertion that these 13 never appear in a training fold.

4. How to read each evaluation figure — for this problem

The figures are standard, but what a given shape *means here* depends on the task: binary classification (functional vs. non-functional), roughly balanced classes (~50/50 by median split), and a headline that is

the contiguous split.

ROC curve

Plots true-positive rate against false-positive rate as you sweep the decision threshold. The diagonal is random. **For us:** a curve that hugs the top-left means the model separates functional from non-functional variants well. Because our classes are balanced, ROC is a fair summary here (it can mislead on imbalanced data, which is why we also show precision-recall). Compare the same model's ROC across the three splits: the contiguous curve dropping toward the diagonal is the leakage tax made visual.

Precision-Recall curve

Plots precision (of the variants I called non-functional, how many were?) against recall (of the truly non-functional, how many did I catch?). The baseline is the class prevalence, not 0.5. **For us:** this matters most if we later shift to an imbalanced label threshold, or if the clinical cost of a false "still resistant" call is high — precision on the non-functional class is what tells you how often a "this variant is dead" call is trustworthy.

Confusion matrix

The raw 2×2 count of true/false positives and negatives at a chosen threshold. **For us:** this is where you see *which kind* of error the model makes — does it miss non-functional variants (dangerous: it would call a dead resistance gene alive) or over-call them? For a resistance tool those two errors have very different consequences, and the matrix is the only figure that separates them explicitly.

Gains & lift charts

Rank all variants by predicted score, then ask: if I only act on the top X%, what fraction of the true positives do I capture (gains), and how many times better than random is that (lift)? **For us:** this is the "prioritization" view — if we could only wet-lab-test a handful of predicted-non-functional variants, gains/lift tells us how much the model concentrates the real hits into the top of the ranking. Directly relevant to picking which mutations to test.

Learning curve

Model score vs. amount of training data, for both training and validation sets. **For us:** it diagnoses whether we are data-limited. If training and validation scores are far apart and the validation curve is still rising, more data would help (a real possibility with ~4,700 variants). If they have converged, the ceiling is the features/model, not the data — which would point us toward richer representations rather than more examples.

Calibration curve

Plots predicted probability against observed frequency; the diagonal is perfect calibration. **For us:** a model can rank variants well (good AUROC) but still output probabilities that don't mean what they say. If a clinical tool reports "80% chance this gene is non-functional," that number needs to be trustworthy. Points below the diagonal = overconfident, above = underconfident. Tree ensembles (RF/XGBoost) are often miscalibrated and may need a calibration step before any probability is reported clinically.

AUROC comparison figure

The bar/point summary of AUROC across every classifier $\times$ feature set $\times$ split. **For us:** this is the scoreboard for the project's thesis. Read it by *split*: within the contiguous panel, does any PLM-derived feature beat identity one-hot, and by how much? That single comparison, on that single split, is the number the whole justification rests on. Every bar is labeled with its exact classifier and exact feature set, so no bar is ambiguous.

5. Why this specific model chain — sequence $\rightarrow$ prediction

The end goal is a clinical tool: sequence in, functional/non-functional out, eventually for double and triple substitutions. The chain is:

1. **PLM reads the sequence** and produces features that encode learned biology (conservation, stability, context).
2. **A tree-based classifier (Random Forest / XGBoost) consumes those features** plus cheap physicochemical signals, and learns — from the DMS labels — how to reweight them toward *TEM-1's specific* functional requirements, including the catalytic positions the raw PLM score under-weights. This is the "the classifier learns the biology the PLM teaches it" step.
3. **The contiguous split certifies** that what it learned transfers to unseen regions, which is the precondition for applying it to a novel gene.
4. **Higher-order substitutions** (double/triple) are the extension: single-substitution DMS is the foundation the combinatorial model builds on, and the feature representations chosen here (especially per-position + physicochemical) are the ones that extend naturally to combinations.

Every choice in phases 0–3 is in service of getting step 2's inputs right and step 3's test honest.